



Universidade de Brasília - UnB
Faculdade de Ciências e Tecnologias em Engenharia - FCTE
Engenharia de Software

Héllen Clícia Barros Rêgo, 251009784;
Italo Souza Moura, 251022418;
Jéssyca Pereira de Souza Farias, 251023246;
Júlia Neiva Lima, 251023291;
Maria Vitória Queiroz Lima, 251013740.

Relatório do trabalho prático de Orientação a Objetos:
Cafeteria Geek "Byte & Brew"

Brasília
2026

O presente relatório tem como objetivo apresentar o projeto de software desenvolvido no âmbito do trabalho prático da disciplina, referente ao sistema de vendas e fidelidade da cafeteria temática Byte & Brew. O propósito central do trabalho consiste na modelagem e no desenvolvimento de um protótipo destinado ao gerenciamento de comandas, fluxo de caixa, controle de estoque do cardápio que é composto por comidas e bebidas/poções, e implementação de um programa gamificado de lealdade denominado de XP do Cliente, aplicando-se os conceitos fundamentais da Orientação a Objetos na linguagem de programação Java.

A arquitetura do sistema foi organizada obrigatoriamente em quatro pacotes distintos:

- br.edu.cafeteria.modelo;
- br.edu.cafeteria.excecao;
- br.edu.cafeteria.servico;
- br.edu.cafeteria.app.

O software implementado em conformidade com as regras de negócio estabelecidas para o domínio será descrito em: descrição das classes, conceitos aplicados e a apresentação do diagrama UML:

1- Descrição das classes

- *Pacote modelo:*

Produto: A classe Produto reúne as características fundamentais que qualquer item do cardápio da cafeteria possa ter, como: código identificador único, nome, preço base e quantidade em estoque, que garantem o monitoramento e a precificação correta dos itens no sistema. Produto é uma classe abstrata, que faz parte do pacote de modelo, pois não existe um produto genérico para ser vendido no balcão: ou ele é uma Comida ou é uma Bebida. Assim, se alguém tentar instanciar um objeto diretamente do tipo Produto, o sistema não irá permitir. Como métodos, além dos acessores encapsulados para proteger os dados, possui o método transacional *reduzirEstoque()*, que diminui automaticamente a quantidade disponível no inventário sempre que uma venda daquele item é concretizada, impedindo que o estoque fique inconsistente.

Comida: A classe Comida é uma subclasse de Produto, que representa os itens comestíveis elaborados na cozinha ou confeitaria da cafeteria, como o *Lembas Bread* e o *Portal Cake*. Possui, além dos atributos herdados de Produto, atributos específicos de restrição alimentar e de preparo, como o *tempoPreparo* em minutos e os indicadores booleanos *vegano* e *semGluten*, que servem para sinalizar se o alimento atende a dietas específicas. Implementa os métodos construtores e de acesso para gerenciar essas características dietéticas, garantindo que o sistema informe adequadamente o tempo de espera e a composição dos pratos.

Bebida: A classe Bebida, também é uma subclasse de Produto, que representa as poções, sodas e cafés servidos no estabelecimento, a exemplo do *Café do Programador* e da *Poção de Mana*. Possui, além dos atributos herdados da superclasse Produto, atributos específicos como o *tamanho* do recipiente, representado por caracteres padronizados como 'P', 'M' ou 'G' e a

quantidadeCafeina em miligramas, que mede o teor de estimulantes na porção. Implementa seus próprios métodos de encapsulamento para garantir que as bebidas sejam sempre servidas na padronização volumétrica correta exigida pela cafeteria.

Atendente: A classe Atendente representa o funcionário responsável pelo balcão e pela abertura das comandas no sistema. Ela reúne características básicas de identificação civil e funcional, como *nome* e *cpf*. A presença dessa classe é indispensável para a dinâmica de atendimento, pois cada pedido aberto precisa ser associado a um objeto Atendente, garantindo o controle interno, a segurança do negócio e a rastreabilidade de quem registrou cada venda na cafeteria.

Cliente: A classe Cliente é abstrata que tem como método *pagar* para receber o pagamento dos clientes das classes ClienteCasual e ClienteCadastrado sem depender do programa de fidelidade.

ClienteCasual: A classe ClienteCasual é herança da classe abstrata Cliente, sobreescrevendo o método da classe mãe para aqueles clientes que não deseja se cadastrar no programa de fidelidade.

ClienteCadastrado: A classe ClienteCadastrado também é herança da classe Cliente e reúne os dados cadastrais básicos que qualquer consumidor participante do programa "XP do Cliente" possa ter, pois, o programa é exclusivo para clientes cadastrados e recebe *nome*, *cpf único* e um *saldoXP*, que acumula os pontos de experiência ganhos nas compras. É uma classe abstrata, pois não existe um cliente cadastrado sem um nível de guilda no sistema: ou ele pertence à categoria iniciante ou ao alto nível, não sendo permitido instanciar um objeto genérico desse tipo. Como métodos, ela define o contrato de acumular pontos por meio de *adicionarXP(double valorCompra)*, além de armazenar as constantes estáticas de conversão de pontos e bônus multiplicadores, que servem de base para que cada subclasse aplique sua própria taxa de recompensa.

ClienteStandard: A classe ClienteStandard é uma subclasse de ClienteFidelidade, que representa o consumidor que está no nível inicial do programa de relacionamento, conhecido como "Aventureiro Iniciante". Possui, além dos atributos herdados de ClienteFidelidade, a responsabilidade de implementar o método *adicionarXP(double valorCompra)*, que aplica a regra de pontuação padrão da cafeteria: a cada R\$ 1,00 gasto em compras, o cliente ganha exatamente 1 ponto de XP. Além disso, essa classe contempla a regra de negócio que permite ao cliente pagar metade, 50%, do valor total da sua compra utilizando o saldo acumulado em pontos de experiência. Também implementa o método exclusivo *pagarComXP(double valorCompra)*, que permite quitar um pedido inteiramente resgatando o saldo de XP com base na taxa constante de conversão onde 10 XP = R\$ 1,00. Caso o cliente tente pagar a compra com pontos mas não possua o saldo suficiente, o método impede a transação e lança a exceção customizada *PontosInsuficientesException*.

ClienteVip: A classe ClienteVIP é uma subclasse de ClienteFidelidade, que representa o consumidor frequente que alcançou o status de lealdade máxima, conhecido como "Mestre da Guilda". Possui como diferencial o polimorfismo por sobrescrita no método *adicionarXP(double valorCompra)*, aplicando um bônus

multiplicador que concede 2 pontos de XP para cada R\$ 1,00 gasto, bonificando o dobro do nível standard. Também implementa o método exclusivo *pagarComXP(double valorCompra)*, que permite quitar um pedido inteiramente resgatando o saldo de XP com base na taxa constante de conversão onde 10 XP = R\$ 1,00. Caso o cliente tente pagar a compra com pontos mas não possua o saldo suficiente, o método impede a transação e lança a exceção customizada *PontosInsuficientesException*.

ItemPedido: Representa a entidade associativa do consumo. Estabelece a ligação direta entre um objeto *Produto* do cardápio e a quantidade inteira solicitada na comanda, contendo a lógica necessária para o cálculo do subtotal financeiro do item.

Pedido: Constitui a entidade central do fluxo operacional de vendas. Cada comanda é gerada com um identificador sequencial único (controlado por um atributo estático auto-incrementado no momento da instancialização), sendo associada a um atendente e, de forma facultativa, a um *Cliente* cadastrado. A classe gerencia uma estrutura de agregação interna (*List<ItemPedido>*). Disponibiliza os métodos *adicionarItem()*, o método de totalização financeira da venda e o método *finalizarPedido()*, que coordena a dedução do estoque dos produtos consumidos e a atribuição dos pontos de experiência ao consumidor, caso exista vínculo cadastral.

- *Pacote Serviço*:

InterfacePromocional: Contrato arquitetural puramente de serviço que estabelece a assinatura da rotina de abatimento financeiro *aplicarDesconto(double valor): double*. Desacopla as políticas sazonais de marketing da estrutura de cardápio fixa.

PromocionalGeek: Implementação concreta da interface *Promocional*. Contém a lógica de negócio responsável por interceptar o valor total de itens enquadrados como bebida em datas festivas como o "Dia do Orgulho Nerd" e aplicar um abatimento automatizado de 10%, retornando o valor precificado ao caixa.

ClienteService: classe prestadora de serviços de gestão para o programa de lealdade. Define o contrato para as rotinas de manipulação de repositório e transações civis, estipulando métodos como *verificarCliente(cpf:String)*, *cadastrear(cpf: String)*, *atualizar(cpf: String)*, *remover(cpf: String)* e *listarTodos(): List*, garantindo o baixo acoplamento entre a persistência dos dados e a lógica transacional do balcão.

- *Pacote exceção*:

EstoqueInsuficienteException: Exceção customizada de verificação obrigatória, lançada pelo sistema em eventos onde a demanda de um produto informada no método *adicionarItem* excede a quantidade matematicamente disponível em vitrine ou vitrine refrigerada.

PontosInsuficienteException: Exceção customizada de verificação obrigatória, ao fluxo de pagamento do perfil Standard e VIP em *pagarXP*. É acionada se, no

momento do encerramento, o saldo acumulado em *saldoXP* for numericamente inferior ao exigido para cobrir o valor integral do pedido na taxa de resgate vigente.

- *Pacote App:*

Main: Classe executável que abriga o método estático principal main(String[] args). Responsável por instanciar os repositórios de serviço, inicializar o cardápio base da Byte & Brew e orquestrar a execução do protótipo no terminal em linha de comando, simulando o fluxo de atendimento contínuo.

2- Conceitos aplicados:

Herança: Princípio empregado para mitigar a redundância de código e instituir uma taxonomia coerente de domínio. Observa-se sua aplicação no cardápio, onde as classes Comida e Bebida derivam da superclasse abstrata Produto, herdando a estrutura comum de precificação, identificação alfanumérica e estoque, ao mesmo tempo em que especializam regras dietéticas e volumétricas. Analogamente, no programa de fidelidade, as classes ClienteStandard e ClienteVIP estendem a superclasse abstrata ClienteCadastrado, reutilizando a identificação civil, o acúmulo de saldo XP e as constantes de conversão.

Associações: Foram parametrizadas para modelar o acoplamento estrutural entre os objetos do sistema:

- **Composição:** O objeto da classe Pedido encapsula e gerencia o ciclo de vida de uma coleção de itens (*List<ItemPedido>*), que representa a comanda de consumo.
- **Associação Unidirecional:** Cada instância de *ItemPedido* aponta diretamente para o insumo da classe *Produto*, enquanto todo objeto *Pedido* detém uma referência obrigatória para o operador funcional da classe Atendente que registrou a venda.
- **Agregação Opcional:** A comanda prevê uma multiplicidade 0..1 com o cadastro do consumidor. Caso o CPF não seja fornecido no terminal, a referência é avaliada como nula, configurando um cliente casual e viabilizando o encerramento da transação monetária em *finalizarPedido()* sem a incidência de ganho ou resgate de experiência, evitando a criação de registros vazios no sistema.

Polimorfismo: A modelagem do sistema comprova a utilização ostensiva das quatro manifestações fundamentais de polimorfismo, em consonância com as exigências curriculares:

- **Polimorfismo por Inclusão:** Verificado no processamento interno da classe *Pedido* e na totalização de itens por meio da classe *ItemPedido*. A rotina de baixa em inventário via *atualizarEstoque()* e o cálculo de subtotal operam sobre coleções que referenciam o tipo superclasse *Produto*.
- **Polimorfismo por Sobrescrita (Overriding):** Evidenciado na especialização comportamental do cálculo de recompensas. O método *xp_a_Receber()*, contratualmente definido na superclasse *ClienteCadastrado* e implementado

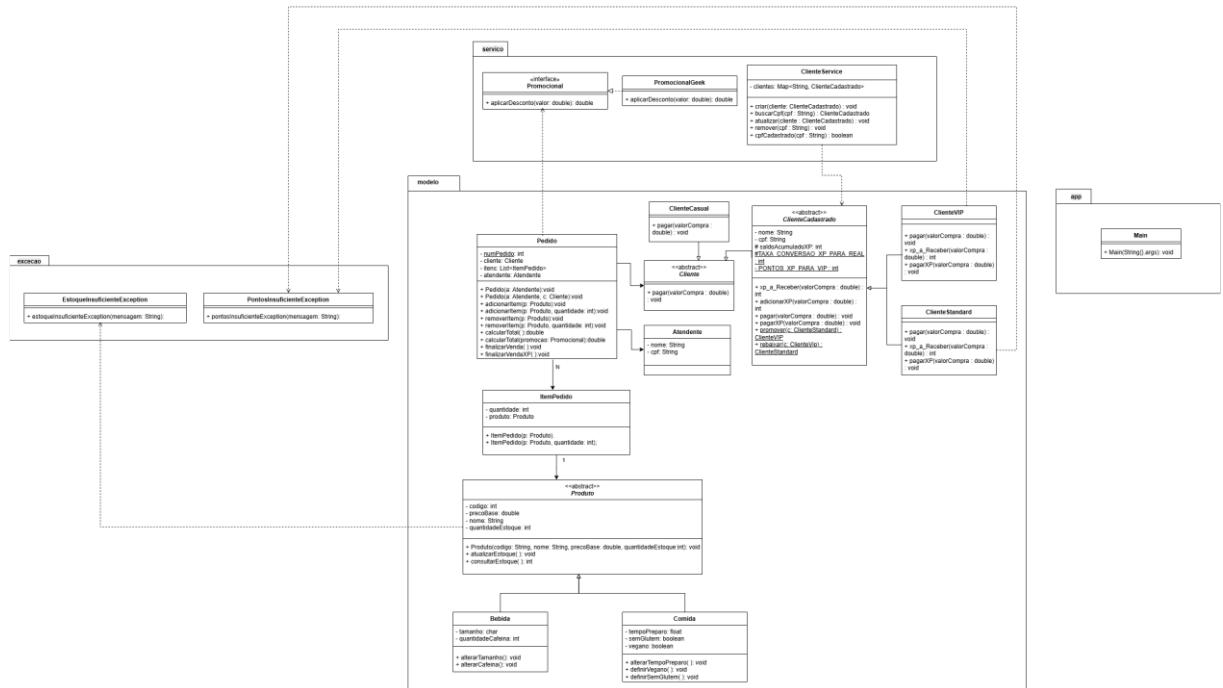
- linearmente na classe *ClienteStandard*, é expressamente redefinido mediante anotação *@Override* na subclasse *ClienteVIP*, alterando a lógica matemática
- para atribuir bonificação dobrada aos membros da guilda, sem alterar a assinatura funcional da chamada.
 - Polimorfismo por Sobrecarga (Overloading): Implementado amplamente na entidade *Pedido* com o intuito de flexibilizar as rotinas de interface de atendimento:
 - a) Sobrecarga de Construtores: O sistema faculta a instancialização da comanda mediante a assinatura *Pedido(Atendente)*, para clientes casuais/anônimos ou via *Pedido(Atendente, ClienteCadastrado)* quando há vinculação com o programa de fidelidade no ato da abertura.
 - b) Sobrecarga de Inserção de Itens: Disponibilização simultânea das assinaturas *adicionarItem(Produto p)*, dmissão unitária implícita, subentendendo quantidade 1 e *adicionarItem(Produto p, int quantidade)*, admissão quantitativa múltipla.
 - c) Sobrecarga de Totalização: Existência de *calcularTotal()* para cômputo convencional e de *calcularTotal(Promocional promocao)* para processamento parametrizado mediante injeção do serviço de abatimento de eventos geeks.
 - Polimorfismo por Coerção: Manifesta-se nos cálculos transacionados nas rotinas do caixa. Ao efetivar a conversão do montante gasto para pontos de experiência no fechamento transacional em *finalizarPedido()*, o valor financeiro contínuo em ponto flutuante é submetido a um processo deliberado de conversão explícita de tipos (type casting) para o formato inteiro (int), adequando a escala monetária à natureza discreta do acúmulo de pontuação sem provocar interrupções de compilação ou perda da estabilidade do sistema.

A modelagem de erros optou pela elaboração de classes de exceção derivando de *RuntimeException*, em oposição às exceções verificadas. No ecossistema Java, tal escolha é justificada pela natureza semântica das falhas operacionais modeladas:

- Natureza de Violação de Estado: Eventos como a falta de ingredientes (*EstoqueInsuficienteException*) ou a tentativa de quitação sem saldo pontuado (*PontosInsuficientesException*) representam violações de regras de negócio e de lógica de domínio, causadas por fluxo inválido ou tentativa de uso indevido da API, e não falhas externas de infraestrutura ou I/O recuperáveis (como a perda de conexão com um banco de dados ou ausência de arquivo físico).
- Filosofia Fail-Fast e Código Limpo: Ao estenderem a classe *RuntimeException*, o sistema adota a premissa de falhar rapidamente ao detectar uma transação inconsistente, sem obrigar a camada de domínio ou as classes intermediárias a poluírem o código com blocos burocráticos de try-catch ou declarações throws em toda a cadeia de chamadas. A responsabilidade pela captura e exibição amigável do erro é concentrada estrategicamente na camada de interface no balcão, gerida pela classe *CafeteriaApp*, preservando a coesão das entidades de negócio.

3- Diagrama de Classes UML:

O diagrama de classes reflete fielmente as diretrizes e a segmentação modular da aplicação da Byte & Brew. Conforme evidencia a representação visual, o design assegura um estrito encapsulamento, em que todos os atributos estruturais são declarados com escopo privado ou protegido, demonstrando o perfeito alinhamento entre a modelagem estática conceitual, a tipagem de identificadores alfanuméricos, a consolidação do serviço concretizado de clientes e a implementação concreta das regras de negócio na linguagem Java.



REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6022**: informação e documentação: artigo em publicação periódica e/ou científica: apresentação. 2. ed. Rio de Janeiro, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6028**: informação e documentação: resumo, resenha e revisão: apresentação. 2. ed. Rio de Janeiro, 2021.

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. **Normas de apresentação tabular**. 3. ed. Rio de Janeiro, 1993.

BARNES, David J.; KÖLLING, Michael. **Programação Orientada a Objetos com Java**: uma introdução prática usando o BlueJ. 4. ed. São Paulo: Pearson Education do Brasil, 2010.

DEITEL, Paul; DEITEL, Harvey. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

GAMMA, Erich et al. **Padrões de Projeto**: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

MARTIN, Robert C. **Código Limpo**: habilidades práticas do Agile Software. Rio de Janeiro: Alta Books, 2009.

ORACLE. **Java Documentation**: Java Platform, Standard Edition 17 API Specification. Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/>. Acesso em: 05 jul. 2026.